- Deprecated the Parallelism knowledge unit and moved parts of its topic to the Basic Concepts knowledge unit.
- Renamed the Reliability through Redundancy knowledge unit to System Reliability.
- Added the Society, Ethics, and the Profession knowledge unit.

Core Hours

Knowledge Unit	CS Core	KA Core
Overview of Computer Systems	3	0
Basic Concepts	4	0
Resource Management	1	1
System Performance	2	2
Performance Evaluation	2	2
System Reliability	2	1
System Security	2	1
System Design	2	1
Society, Ethics, and Profession	Included in SEP hours	
Total	**18**	**8**

Knowledge Units

SF-Overview: Overview of Computer Systems

***CS Core**:*

1. Basic building blocks and components of a computer (gates, flip-flops, registers, interconnections; datapath + control + memory)
2. Hardware as a computational paradigm: Fundamental logic building blocks; Logic expressions, minimization, sum of product forms (See also: AR-Logic)
3. Programming abstractions, interfaces, use of libraries (See also: PDC-Programs)
4. Distinction and interaction between application and OS services, remote procedure call (See also: OS-Purpose)
5. Basic concept of pipelining, overlapped processing stages (See also: AR-Organization)
6. Basic concept of scaling: performance vs problem size

Illustrative Learning Outcomes:
***CS Core**:*

1. Describe the basic building blocks of computers and their role in the historical development of computer architecture.
2. Design a simple logic circuit using the fundamental building blocks of logic design to solve a simple problem (e.g., adder).
3. Describe how computing systems are constructed of layers upon layers, based on separation of concerns, with well-defined interfaces, hiding details of low layers from the higher layers.
4. Describe that hardware, OS, VM, and application are additional layers of interpretation/processing.
5. Describe the mechanisms of how errors are detected, signaled back, and handled through the layers.
6. Construct a simple program (e.g., a TCP client/server) using methods of layering, error detection and recovery, and reflection of error status across layers.
7. Identify bugs in a layered program by using tools for program tracing, single stepping, and debugging.
8. Understand the concept of strong vs weak scaling, i.e., how performance is affected by the scale of the problem vs the scale of resources to solve the problem. This can be motivated by simple, real-world examples.

SF-Foundations: Basic Concepts

CS Core:

1. Digital vs Analog/Discrete vs Continuous Systems
2. Simple logic gates, logical expressions, Boolean logic simplification
3. Clocks, State, Sequencing
4. State and state transition (e.g., starting state, final state, life cycle of states) (See also: AL-Models)
5. Finite state machines (e.g., NFA, DFA) (See also: AL-Models)
6. Combinational Logic, Sequential Logic, Registers, Memories (See also: AR-Logic)
7. Computers and Network Protocols as examples of State Machines (See also: NC-Fundamentals)
8. Sequential vs parallel processing. (See also: PDC-Programs, OS-Concurrency)
9. Application-level sequential processing: single thread (See also: PDC-Programs, OS-Concurrency)
10. Simple application-level parallel processing: request level (web services/client-server/distributed), single thread per server, multiple threads with multiple servers, pipelining (See also: PDC-Programs, OS-Concurrency)

Illustrative Learning Outcomes:

CS Core*:*

1. Describe the differences between digital and analog systems, and between discrete and continuous systems. Can give real-world examples of these systems.
2. Describe computations as a system characterized by a known set of configurations with transitions from one unique configuration (state) to another (state).
3. Describe the distinction between systems whose output is only a function of their input (stateless) and those with memory/history (stateful).
4. Develop state machine descriptions for simple problem statement solutions (e.g., traffic light sequencing, pattern recognizers).
5. Describe a computer as a state machine that interprets machine instructions.

6. Explain how a program or network protocol can also be expressed as a state machine and that alternative representations for the same computation can exist.
7. Derive the time-series behavior of a state machine from its state machine representation (e.g., TCP connection management state machine).
8. Write a simple sequential problem and a simple parallel version of the same program.
9. Evaluate the performance of simple sequential and parallel versions of a program with different problem sizes and be able to describe the speed-ups achieved.
10. Describe on an execution timeline how parallelism events and operations can take place simultaneously (i.e., at the same time). Explain how work can be performed in less elapsed time if this can be exploited.

SF-Resource: Resource Management

***CS Core**:*

1. Different types of resources (e.g., processor share, memory, disk, net bandwidth) (See also: OS-Scheduling, OS-Memory, OS-Files, NC-Fundamentals)
2. Common resource allocation/scheduling algorithms (e.g., first-come-first-serve, priority-based scheduling, fair scheduling, and preemptive scheduling) (See also: OS-Scheduling)

KA Core:

3. Advantages and disadvantages of common scheduling algorithms (See also: OS-Scheduling)

***Illustrative Learning Outcomes**:*

CS Core:

1. Define how finite computer resources (e.g., processor share, memory, storage, and network bandwidth) are managed by their careful allocation to existing entities.
2. Describe how common resource allocation/scheduling algorithms work.
3. Develop common scheduling algorithms and evaluate their performances.

KA Core:

4. Describe the pros and cons of common scheduling algorithms.

SF-Performance: System Performance

***CS Core**:*

1. Latencies in computer systems
    a. Speed of light and computers (one foot per nanosecond vs one GHz clocks) (See also: AR-Organization)
    b. Memory vs disk latencies vs across-the-network memory (See also: AR-Memory, AR-Performance-Energy)
2. Caches and the effects of spatial and temporal locality on performance in processors and systems (See also: AR-Memory, AR-Performance-Energy, OS-Memory)
3. Caches and cache coherency in databases, operating systems, distributed systems, and computer architecture (See also: OS-Memory, AR-Memory, DM-Internals)
4. Introduction to the processor memory hierarchy (See also: AR-Memory, AR-Performance-Energy)

KA Core:

5. The formula for average memory access time (See also: AR-Memory)
6. Rationale of virtualization and isolation: protection and predictable performance (See also: OS-Virtualization)
7. Levels of indirection, illustrated by virtual memory for managing physical memory resources (See also: OS-Virtualization)
8. Methods for implementing virtual memory and virtual machines (See also: OS-Virtualization)

Illustrative Learning Outcomes*:*

CS Core:

1. Describe the breakdown of the latency of computer systems in terms of memory, disk, and network.
2. Explain the importance of locality in determining system performance.
3. Calculate average memory access time and describe the tradeoffs in memory hierarchy performance in terms of capacity, miss/hit rate, and access time.

KA Core:

4. Explain why it is important to isolate and protect the execution of individual programs and environments that share common underlying resources.
5. Describe how the concept of indirection can create the illusion of a dedicated machine and its resources even when physically shared among multiple programs and environments.
6. Evaluate the performance of two application instances running on separate virtual machines and determine the effect of performance isolation.

SF-Evaluation: Performance Evaluation

CS Core*:*

1. Performance figures of merit (See also: AR-Performance-Energy, PDC-Evaluation)
2. Workloads and representative benchmarks, and methods of collecting and analyzing performance figures of merit (See also: AR-Performance-Energy, PDC-Evaluation)
3. CPI (Cycles per Instruction) equation as a tool for understanding tradeoffs in the design of instruction sets, processor pipelines, and memory system organizations (See also: AR-Performance-Energy, PDC-Evaluation)
4. Amdahl's Law: the part of the computation that cannot be sped up limits the effect of the parts that can (See also: AR-Performance-Energy, PDC-Evaluation)
5. Order of magnitude analysis (Big O notation) (See also: AL-Complexity)
6. Analysis of slow and fast paths of a system (See also: AR-Organization)
7. Events on their effect on performance (e.g., instruction stalls, cache misses, page faults) (See also: OS-Memory, AR-Organization)

KA Core:

7. Analytical tools to guide quantitative evaluation
8. Understanding layered systems, workloads, and platforms, their implications for performance, and the challenges they represent for evaluation
9. Microbenchmark pitfalls

***Illustrative Learning Outcomes**:*

CS Core:

1. Explain how the components of system architecture contribute to improving its performance.
2. Explain the circumstances in which a given figure of a system performance metric is useful.
3. Explain the usage and inadequacies of benchmarks as a measure of system performance.
4. Describe Amdahl's law and discuss its limitations.
5. Apply limit studies or simple calculations to produce order-of-magnitude estimates for a given performance metric in a given context.
6. Apply software tools to profile and measure program performance.

KA Core:

7. Design and conduct a performance-oriented experiment of a common system (e.g., an OS and Spark).
8. Design a performance experiment on a layered system to determine the effect of a system parameter on system performance.

SF-Reliability: System Reliability

***CS Core**:*

1. Distinction between bugs, faults, and failures (See also: PDC-Coordination, SE-Reliability)
2. Reliability vs availability
3. Reliability through redundancy
   a. check and retry (See also: OS-Faults, NC-Reliability)
   b. redundant encoding (error correction codes, CRC, FEC, RAID) (See also: AR-Memory, NC-Reliability, DM-Distributed)
   c. duplication/mirroring/replicas (See also: DM-Distributed)

KA Core:

4. Other approaches to reliability (e.g., journaling) (See also: OS-Faults, NC-Reliability, SE-Reliability)

***Illustrative Learning Outcomes**:*

CS Core:

1. Explain the distinction between program errors, system errors, and hardware faults (e.g., corrupted memory) and exceptions (e.g., attempt to divide by zero).
2. Articulate the distinction between detecting, handling, and recovering from faults and the methods for their implementation.
3. Describe the role of error correction codes in providing error checking and correction techniques in memories, storage, and networks.
4. Apply simple algorithms for exploiting redundant information for the purposes of data correction.

KA Core:

5. Compare different error detection and correction methods for their data overhead, implementation complexity, and relative execution time for encoding, detecting, and correcting errors.

SF-Security: System Security

***CS Core**:*

1. Common system security issues (e.g., viruses, denial-of-service attacks, and eavesdropping) (See also: OS-Protection, NC-Security, SEC-Foundations, SEC-Engineering)
2. Countermeasures (See also: OS-Principles, OS-Protection, NC-Security)
   a. Cryptography (See also: SEC-Crypto)
   b. Security architecture (See also: SEC-Engineering)

KA Core:

3. Representative countermeasure systems
   a. Intrusion detection systems, firewalls (See also: NC-Security)
   b. Antivirus systems

***Illustrative Learning Outcomes**:*

***CS Core**:*

1. Describe some common system security issues and give examples
2. Describe some countermeasures against system security issues

***KA Core**:*

3. Describe representative countermeasure systems

SF-Design: System Design

***CS Core**:*

1. Common criteria of system design (e.g., liveness, safety, robustness, scalability, and security) (See also: PDC-Evaluation)

KA Core:

2. Designs of representative systems (e.g., Apache web server, Spark, and Linux)

***Illustrative Learning Outcomes**:*

CS Core:

1. Describe common criteria of system design.
2. Given the functionality requirements of a system and its key design criteria, provide a high-level design of this system.

KA Core:

3. Describe the design of some representative systems.

SF-SEP: Society, Ethics, and the Profession

***KA Core**:*

1. Intellectual property rights of computer systems (See also: SEP-IP)
2. Common software licenses (See also: SEP-IP)
3. Computer crimes (See also: SEP-Security)

***Illustrative Learning Outcomes**:*
***KA Core**:*

1. Describe the intellectual property rights of computer systems.
2. List representative software licenses and compare their differences.
3. List representative computer crimes.

Professional Dispositions

- **Meticulous**: Students must pay attention to details of different perspectives when learning about and evaluating systems.
- **Adaptive**: Students must be flexible and adaptive when designing systems. Different systems have different requirements, constraints and working scenarios. As such, they require different designs. Students must be able to make appropriate design decisions correspondingly.

Mathematics Requirements

Required:

- Discrete Mathematics (See also: MSF-Discrete)
  - Sets and relations
  - Basic graph theory
  - Basic logic
- Linear Algebra (See also: MSF-Linear)
  - Basic matrix operations
- Probability and Statistics (See also: MSF-Probability, MSF-Statistics)
  - Random variable
  - Bayes theorem
  - Expectation and Variation
  - Cumulative distribution function and probability density function

Desirable:

- Basic queueing theory
- Basic stochastic process

Course Packaging Suggestions

Introductory Course to include the following:

- SF-Overview (2 hours)
- SF-Foundations (6 hours)
- SF-Resource (4 hours)
- SF-Performance (6 hours)
- SF-Evaluation (6 hours)
- SF-Reliability (4 hours)